

打刻内容編集申請画面の表示の修正

この「カレンダーの月次移動制限」および「過去日の編集申請期間制限」は、システムの悪用を防ぎ、労務管理データの一貫性を保つために極めて重要な運用統制（ガバナンス）の設計です。

1. 未来や過去のデータへ際限なくアクセスできてしまう状態を防ぐ（前後3ヶ月制限）
2. 締め処理等の運用を考慮し、過去の打刻修正は直近1ヶ月以内に制限する（1ヶ月前まで制限）

このルールをフロントエンドの見た目（UX）だけでなく、バックエンドの防壁（バリデーション）の双方に組み込むことで、システムはより強固になります。

まずは**バックエンド側の修正内容とロジックの解説**から進めていきましょう。

バックエンド側での規制トラフィックの考察

フロントエンドが「前後3ヶ月しか表示しない」「1ヶ月前までしか申請ボタンを出さない」という制御を行っても、悪意あるユーザーがAPIのURLを直接叩く（ツール等で直接リクエストを送る）可能性を考慮しなければなりません。そのため、バックエンド側で「二重の防壁」を構築します。

今回修正対象となるのは、打刻修正申請の窓口である `AttendanceEditController.java` です。

【バックエンドがジャッジする厳格なルール】

- ① 申請しようとしている対象日（recordDate）が、操作日（今日）から「1ヶ月前」より古い過去なら却下。
 - ② 申請しようとしている対象日（recordDate）が、操作日（今日）から「3ヶ月後」より未来なら却下。
- ※ただし、操作日当日の申請（退勤未打刻状態での申請など）は、これまで通り優しく通します。

バックエンド：AttendanceEditController.java の修正コード

以前に当日空欄申請のためにアップデートした `@PostMapping("/request-edit")` メソッドのバリデーションブロックに、「今日から起算した期間制限の防壁」を新しく注入します。

修正・差し替えコード

`AttendanceEditController.java` の該当メソッド内を、以下のようにアップデートしてください。

`java.time`パッケージの `minusMonths` や `plusMonths` を用いることで、月を跨いだ日付計算も安全かつ正確に行えます。

```
/**
 * 窓口2: ユーザーからの打刻修正（編集）申請リクエストを受け付ける
 * POST http://localhost:8080/api/attendance/request-edit
 */
@PostMapping("/request-edit")
public ResponseEntity<String> requestEdit(@RequestBody com.appspace.backend.dto.TimeChangeRequest request) {

    // 1. 備考（修正理由）の空っぽチェック
    if (request.getReason() == null || request.getReason().trim().isEmpty()) {
        return ResponseEntity.badRequest().body("【却下】修正理由（備考）の入力は必須です。");
    }
}
```

```

// 2. 日付判定の準備
java.time.LocalDate today = java.time.LocalDate.now();
java.time.LocalDate targetDate = request.getRecordDate();
boolean isToday = targetDate.equals(today);

// =====
// 【新規追加：期間統制の二重防壁】
// =====
if (!isToday) {
    // 制限1: 操作日当日以外の場合、申請ができるのは「当日の1ヶ月前」の同日まで
    java.time.LocalDate oneMonthAgo = today.minusMonths(1);
    if (targetDate.isBefore(oneMonthAgo)) {
        return ResponseEntity.badRequest().body("【却下】過去の打刻内容編集申請は、1ヶ月前のシステム許容期間内の日付に限られます。");
    }

    // 制限2: 安全策として、表示上限である「3ヶ月後」を超える未来の申請もブロック
    java.time.LocalDate threeMonthsLater = today.plusMonths(3);
    if (targetDate.isAfter(threeMonthsLater)) {
        return ResponseEntity.badRequest().body("【却下】未来すぎる日付への申請は受け付けられません。");
    }
}
// =====

// 3. 時刻形式チェック用の正規表現 (hh:mm:ss)
String timeRegex = "^[01][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9]$";

// --- Entry (出勤) 時刻のチェック ---
if (request.getTargetEntryTime() == null ||
    request.getTargetEntryTime().equals("-") ||
    request.getTargetEntryTime().trim().isEmpty()) {
    return ResponseEntity.badRequest().body("【却下】出勤時刻(Entry)を入力してください。");
}

```

```

if (!request.getTargetEntryTime().matches(timeRegex)) {
    return ResponseEntity.badRequest().body("【却下】Entry時刻の形式が不正です。(hh:mm:ssで入力してください)");
}

// --- Exit (退勤) 時刻のチェック ---
String exitTime = request.getTargetExitTime();
if (exitTime == null || exitTime.trim().isEmpty()) {
    exitTime = "-";
}

if (isToday) {
    // 当日の場合: 退勤時刻が「-」であっても許容
    if (!exitTime.equals("-") && !exitTime.matches(timeRegex)) {
        return ResponseEntity.badRequest().body("【却下】当日の退勤時刻(Exit)の形式が不正です。");
    }
} else {
    // 前日以前(過去)の場合: 退勤時刻「-」は不可
    if (exitTime.equals("-")) {
        return ResponseEntity.badRequest().body("【却下】過去の日付の申請には、退勤時刻(Exit)の入力も必須です。");
    }
    if (!exitTime.matches(timeRegex)) {
        return ResponseEntity.badRequest().body("【却下】Exit時刻の形式が不正です。(hh:mm:ssで入力してください)");
    }
}

try {
    calendarService.requestTimeChange(
        request.getAccountId(),
        request.getRecordDate(),
        request.getTargetEntryTime(),
        exitTime,
        request.getReason()
    );

    System.out.println("=== [Controller] 期間統制チェックを通過し、修正申請を受付完了 ===");
    return ResponseEntity.ok("打刻内容の修正申請を提出しました。管理者の承認をお待ちください。");
} catch (Exception e) {
    return ResponseEntity.internalServerError().body("申請処理中に重大なエラーが発生しました: " + e.getMessage());
}
}

```

コードの解説と安心ポイント

- `today.minusMonths(1)` の賢さ

Javaの `LocalDate` は非常に賢く、例えば操作日が「3月31日」の場合、1ヶ月前を `minusMonths(1)` で計算すると、存在しない「2月31日」ではなく、自動的に2月の末日（2月28日または29日）に補正して判定してくれます。これにより、月末・月初に関わるバグの発生を防ぐことができます。

- Repository層の修正が不要な理由

今回はデータベースにデータを「保存する手前」で期間の正当性を検証して弾くロジックであるため、データを取得・保存するRepositoryやService側の基本構造を壊すことなく、安全に防壁を追加できています。

次のステップ（フロントエンド） への見通し

バックエンドの防壁はこれで完成です！Javaファイルを保存し、念のため一度コンパイル・起動が正常に通るか確認してみてください。

これが確認できたら、いよいよ本難所である**フロントエンド（Vue.js）側の改修**へ移りましょう。

フロント側（`AttendanceEditView.vue`）では、以下のようなスマートな制御を組み込みます。

1. カレンダーの「前月」ボタンを押した際、もし「今日の3ヶ月前の月」に到達していたらボタンを `disabled`（非活性）にしてそれ以上進めなくする。
2. 「次月」ボタンも同様に、「今日の3ヶ月後の月」に到達したら非活性にする。
3. 表示されているカレンダーの日付リストをループ処理する際、今日から「1ヶ月前」より古い日付の行には、ボタンを非表示、あるいは「申請期間外」という優しいテキストを表示する。

フロントエンドファイル AttendanceEditView.vue の修正

以下の記述で、手元の AttendanceEditView.vue ファイルを丸ごと上書きアップデートしてください。

```
<template>
  <div class="attendance-edit-container">
    <h2>打刻内容編集申請</h2>
    <p class="subtitle">当月1か月分の打刻履歴の確認と、修正申請が行えます。</p>

    <div class="month-selector">
      <button
        @click="changeMonth(-1)"
        :disabled="isMinMonthReached"
        class="btn-arrow"
      >
        ◀ 前月
      </button>

      <span class="current-month-text">{{ currentYearMonth }}</span>
```



```

<button
  @click="changeMonth(1)"
  :disabled="isMaxMonthReached"
  class="btn-arrow"
>
  次月 ▶
</button>
</div>
<div class="calendar-list-wrapper">
  <table class="calendar-table">
    <thead>
      <tr>
        <th>日付</th>
        <th>確定 Entry</th>
        <th>確定 Exit</th>
        <th>申請状態</th>
        <th>操作</th>
      </tr>
    </thead>

    <tbody>
      <tr v-for="day in calendarList" :key="day.recordDate" :class="getRowClass(day.recordDate)">
        <td class="date-col">{{ formatDate(day.recordDate) }}</td>

        <td><span class="time-text">{{ day.entryTime }}</span></td>
        <td><span class="time-text">{{ day.exitTime }}</span></td>

        <td>
          <span v-if="day.isTimechangeDemand === 1" class="status-badge" :class="getStatusClass(day.timechangeStatus)">
            {{ getStatusLabel(day.timechangeStatus) }}
          </span>
          <span v-else class="status-none">-</span>
        </td>
      </tr>
    </tbody>
  </table>
</div>

```

```

<td>
  <span v-if="isFutureDate(day.recordDate)" class="text-muted">不可</span>

  <div v-else class="action-buttons-gap">
    <template v-if="isToday(day.recordDate) || isWithinOneMonth(day.recordDate)">
      <button @click="openEditForm(day)" class="btn-action-edit">
        {{ day.isTimechangeDemand === 1 ? '再申請' : '編集申請' }}
      </button>

      <button
        v-if="day.isTimechangeDemand === 1 && day.timechangeStatus === 0"
        @click="cancelRequest(day)"
        class="btn-action-cancel"
      >
        取消
      </button>
    </template>

    <span v-else class="text-muted-expired">申請期間外</span>
  </div>
</td>
</tr>
</tbody>
</table>
</div>

<div v-if="showForm" class="edit-form-overlay" @click.self="closeForm">
  <div class="edit-form-card">
    <h3>{{ formatDate(selectedDay.recordDate) }} の修正申請</h3>

```

```
<div class="form-body">
  <div class="form-group">
    <label>修正後の Entry 時刻 (hh:mm:ss)</label>
    <input
      v-model="form.entryTime"
      type="text"
      placeholder="例: 09:00:00"
      maxlength="8"
    />
  </div>

  <div class="form-group">
    <label>修正後の Exit 時刻 (hh:mm:ss)</label>
    <input
      v-model="form.exitTime"
      type="text"
      placeholder="例: 18:00:00"
      maxlength="8"
    />
  </div>

  <div class="form-group">
    <label>備考・修正理由 <span class="required">※必須</span></label>
    <textarea
      v-model="form.reason"
      placeholder="例: 出勤時に打刻を失念したため。〇〇クライアント直行のため。"
      rows="3"
    ></textarea>
  </div>

  <div v-if="selectedDay.timechangeStatus === 1 && selectedDay.adminComment" class="admin-comment-box">
    <strong>管理者からの差戻理由:</strong>
    <p>{{ selectedDay.adminComment }}</p>
  </div>
</div>
```

```

    <div class="form-actions">
      <button @click="submitRequest" class="btn-submit">申請を提出する</button>
      <button @click="closeForm" class="btn-cancel">キャンセル</button>
    </div>
  </div>
</div>
</template>

<script setup>
import { onMounted, ref, computed } from 'vue'; // 💡 computed を追加インポート
import apiClient from '../api';

// 状態管理変数
const calendarList = ref([]);
const currentYearMonth = ref(''); // "2026-05" 形式で保持
const loggedInAccountId = ref('');

// フォーム用のリアクティブ変数
const showForm = ref(false);
const selectedDay = ref(null);
const form = ref({
  entryTime: '',
  exitTime: '',
  reason: ''
});

// 初期設定（現在の年月をセット）
const initCurrentMonth = () => {
  const now = new Date();
  const year = now.getFullYear();
  const month = String(now.getMonth() + 1).padStart(2, '0');
  currentYearMonth.value = `${year}-${month}`;
};

```

```

// 1か月分のカレンダーリストをバックエンドから取得する関数
const fetchMonthlyData = async () => {
  if (!loggedInAccountId.value) return;
  try {
    const response = await apiClient.get('/attendance/monthly-list', {
      params: {
        accountId: loggedInAccountId.value,
        yearMonth: currentYearMonth.value
      }
    });
    calendarList.value = response.data;
  } catch (error) {
    console.error('カレンダーデータの取得に失敗しました:', error);
  }
};

// =====
// 【新規追加】月次移動制限（前後3ヶ月）のcomputed判定ロジック
// =====

// "yyyy-MM" の文字列をベースに、今月（今日）から何ヶ月離れているかを割り出す共通ヘルパー
const getMonthOffsetFromToday = (yearMonthStr) => {
  if (!yearMonthStr) return 0;
  const [targetYear, targetMonth] = yearMonthStr.split('-').map(Number);

  const today = new Date();
  const currentYear = today.getFullYear();
  const currentMonth = today.getMonth() + 1; // 1～12

  // 離れている月数を計算（例：2026-08 から 2026-05 を引くと +3）
  return (targetYear - currentYear) * 12 + (targetMonth - currentMonth);
};

// 過去3ヶ月前に達しているか判定
const isMinMonthReached = computed(() => {
  return getMonthOffsetFromToday(currentYearMonth.value) <= -3;
});

// 未来3ヶ月後に達しているか判定
const isMaxMonthReached = computed(() => {
  return getMonthOffsetFromToday(currentYearMonth.value) >= 3;
});

```

```
// 月の切り替え（前月 / 次月）
const changeMonth = (offset) => {
  // 安全策: 無効なボタン状態でのクリックを関数側でも弾く
  if (offset === -1 && isMinMonthReached.value) return;
  if (offset === 1 && isMaxMonthReached.value) return;

  const [year, month] = currentYearMonth.value.split('-').map(Number);
  const date = new Date(year, month - 1 + offset, 1);
  const nextYear = date.getFullYear();
  const nextMonth = String(date.getMonth() + 1).padStart(2, '0');
  currentYearMonth.value = `${nextYear}-${nextMonth}`;
  fetchMonthlyData(); // 月が変わったらデータを再読み込み
};

// 編集フォームを開く処理
const openEditForm = (day) => {
  selectedDay.value = day;
  form.value.entryTime = day.isTimechangeDemand === 1 ? day.tmpEntryTime : (day.entryTime === '-' ? '09:00:00' : day.entryTime);
  form.value.exitTime = day.isTimechangeDemand === 1 ? day.tmpExitTime : (day.exitTime === '-' ? '18:00:00' : day.exitTime);
  form.value.reason = day.reason || '';
  showForm.value = true;
};

const closeForm = () => {
  showForm.value = false;
  selectedDay.value = null;
};
```

```
// =====
// 日付判定用のユーティリティ関数群
// =====

// 当日であるかを判定する
const isToday = (dateStr) => {
  const todayStr = new Date().toISOString().split('T')[0];
  return dateStr === todayStr;
};

// 翌日以降（未来）であるかを判定する
const isFutureDate = (dateStr) => {
  const todayStr = new Date().toISOString().split('T')[0];
  return dateStr > todayStr;
};

// 【新規追加】操作日当日以外で「今日からちょうど1ヶ月前以内」に収まっているかを精密ジャッジ
const isWithinOneMonth = (dateStr) => {
  const targetDate = new Date(dateStr);
  targetDate.setHours(0, 0, 0, 0);

  const today = new Date();
  today.setHours(0, 0, 0, 0);

  // 今日の日付からジャスト1ヶ月前の同日を算出
  const oneMonthAgo = new Date();
  oneMonthAgo.setMonth(today.getMonth() - 1);
  oneMonthAgo.setHours(0, 0, 0, 0);

  // 対象の日付が「1ヶ月前の同日以降」かつ「今日より前（過去）」である場合のみtrue
  return targetDate >= oneMonthAgo && targetDate < today;
};
```

```

// =====
// 申請提出時のバリデーションロジック
// =====
const submitRequest = async () => {
  const targetDate = selectedDay.value.recordDate;

  if (!form.value.reason.trim()) {
    alert('修正理由（備考）を入力してください。');
    return;
  }

  const timeRegex = "^([01][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9]$";

  if (isToday(targetDate)) {
    if (!form.value.entryTime.match(timeRegex)) {
      alert('当日の申請には、正しい出勤時刻(Entry)の入力が必要です。');
      return;
    }
    if (form.value.exitTime.trim() == '') form.value.exitTime = '-';
    if (form.value.exitTime !== '-' && !form.value.exitTime.match(timeRegex)) {
      alert('退勤時刻(Exit)の形式が不正です。');
      return;
    }
  } else {
    if (!form.value.entryTime.match(timeRegex) || !form.value.exitTime.match(timeRegex)) {
      alert('前日以前の申請を行うには、出勤(Entry)と退勤(Exit)の両方の時刻を入力してください。');
      return;
    }
  }
}

if (!confirm('この内容で打刻内容の編集申請を提出してもよろしいですか？')) {
  return;
}

try {
  await apiClient.post('/attendance/request-edit', {
    accountId: loggedInAccountId.value,
    recordDate: targetDate,
    targetEntryTime: form.value.entryTime,
    targetExitTime: form.value.value.exitTime,
    reason: form.value.reason
  });

  alert('申請が正常に提出されました！');
  closeForm();
  fetchMonthlyData();
} catch (error) {
  console.error('申請エラー:', error);
  alert(error.response?.data || '申請の提出に失敗しました。');
}
};

```



```
// 申請取り消し処理
const cancelRequest = async (day) => {
  if (!confirm(`${formatDate(day.recordDate)} の編集申請を取り消しますか?`)) {
    return;
  }

  try {
    await apiClient.post('/attendance/cancel-edit', {
      accountId: loggedInAccountId.value,
      recordDate: day.recordDate
    });

    alert('申請を取り消しました。');
    fetchMonthlyData();
  } catch (error) {
    console.error('取消エラー:', error);
    alert('申請の取り消しに失敗しました。');
  }
};

// 補助・装飾用ユーティリティ関数群
const formatDate = (dateStr) => {
  if (!dateStr) return '';
  const [y, m, d] = dateStr.split('-');
  return `${Number(m)}月${Number(d)}日`;
};

const getStatusLabel = (status) => {
  if (status === 0) return '申請中 (未承認)';
  if (status === 1) return '差戻し';
  if (status === 2) return '承認済み';
  return '不明';
};

const getStatusClass = (status) => {
  if (status === 0) return 'badge-pending';
  if (status === 1) return 'badge-rejected';
  if (status === 2) return 'badge-approved';
  return '';
};
```

```

const getRowClass = (dateStr) => {
  if (!dateStr) return '';
  const dayNum = new Date(dateStr).getDay();
  if (dayNum === 6) return 'row-saturday';
  if (dayNum === 0) return 'row-sunday';
  return '';
};

onMounted(() => {
  initCurrentMonth();
  const userData = localStorage.getItem('user');
  if (userData) {
    const user = JSON.parse(userData);
    loggedInAccountId.value = user.accountId;
    fetchMonthlyData();
  } else {
    alert('ユーザー情報が取得できません。再ログインしてください。');
  }
});
</script>

<style scoped>
.attendance-edit-container {
  max-width: 900px;
  margin: 40px auto;
  padding: 20px;
  font-family: sans-serif;
  color: #333;
}
.subtitle { color: #666; margin-bottom: 30px; }

/* 月選択のデザイン */
.month-selector {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 20px;
  margin-bottom: 20px;
  background-color: #343a40; padding: 10px; border-radius: 6px;
}

```

```
.btn-arrow {
  background-color: #6c757d;
  border: 1px solid #5a6268;
  color: #ffffff;
  padding: 8px 16px;
  border-radius: 4px;
  cursor: pointer;
  font-weight: bold;
  transition: background-color 0.2s, border-color 0.2s;
}

.btn-arrow:hover:not(:disabled) {
  background-color: #5a6268;
  border-color: #4e555b;
}

/* 変更5: ボタンが非活性 (disabled) になった際の、視覚的にわかりやすいデザイン */
.btn-arrow:disabled {
  background-color: #495057;
  color: #868e96;
  border-color: #495057;
  cursor: not-allowed;
  opacity: 0.6;
}

.current-month-text {
  font-size: 1.3rem;
  font-weight: bold;
  min-width: 120px;
  text-align: center;
  color: #e0e0e0;
  text-shadow: 0 1px 2px rgba(0, 0, 0, 0.2);
}
```

```
/* テーブルカレンダーのデザイン */
.calendar-list-wrapper {
  background: white;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.08);
  overflow: hidden;
}
.calendar-table { width: 100%; border-collapse: collapse; text-align: center; }
.calendar-table th {
  background-color: #f8f9fa;
  color: #495057;
  padding: 14px;
  font-weight: 600;
  border-bottom: 2px solid #dee2e6;
}
.calendar-table td { padding: 12px; border-bottom: 1px solid #eceeef; vertical-align: middle; }

.row-saturday { background-color: #f7faff; }
.row-sunday { background-color: #fff7f7; }
.date-col { font-weight: bold; color: #455a64; }
.time-text { font-family: monospace; font-size: 1rem; }

/* ステータスバッジの装飾 */
.status-badge { display: inline-block; padding: 4px 10px; border-radius: 12px; font-size: 0.85rem; font-weight: bold; }
.badge-pending { background-color: #fff3e0; color: #f57c00; }
.badge-rejected { background-color: #ffebee; color: #d32f2f; }
.badge-approved { background-color: #e8f5e9; color: #388e3c; }
.status-none { color: #ccc; }

.btn-action-edit {
  background-color: #ffffff;
  border: 1px solid #007bff;
  color: #007bff;
  padding: 6px 12px;
  border-radius: 4px;
  cursor: pointer;
  transition: all 0.2s;
}
.btn-action-edit:hover { background-color: #007bff; color: white; }
```

```
/* 変更6: 申請期間を過ぎた過去日用の落ち着いたテキストデザイン */
```

```
.text-muted-expired {  
  color: #90a4ae;  
  font-size: 0.85rem;  
  font-style: italic;  
  font-weight: bold;  
}
```

```
/* 編集申請ポップアップフォームの装飾 */
```

```
.edit-form-overlay {  
  position: fixed;  
  top: 0; left: 0;  
  width: 100%;  
  height: 100%;  
  background: rgba(0,0,0,0.4);  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  z-index: 1000;  
}  
.edit-form-card {  
  background: white;  
  padding: 30px;  
  border-radius: 8px;  
  width: 100%;  
  max-width: 450px;  
  box-shadow: 0 10px 25px rgba(0,0,0,0.15);  
  animation: fadeIn 0.2s ease-out;  
}
```

```
@keyframes fadeIn {
  from { transform: translateY(10px); opacity: 0; }
  to { transform: translateY(0); opacity: 1; }
}

.form-body { margin: 20px 0; text-align: left; }
.form-group { margin-bottom: 18px; }
.form-group label {
  display: block;
  margin-bottom: 6px;
  font-weight: bold;
  font-size: 0.9rem;
  color: #555;
}
.form-group input[type="text"], .form-group textarea {
  width: 100%;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box;
  font-size: 1rem;
}
.form-group input[type="text"] { font-family: monospace; }
.required { color: #d32f2f; }

.admin-comment-box {
  border-left: 4px solid #f57c00;
  padding: 10px;
  margin-top: 15px;
  border-radius: 4px;
  font-size: 0.9rem;
  background: #fff8f0;
}

.form-actions { display: flex; gap: 10px; margin-top: 25px; }
.btn-submit {
  flex: 1;
  background-color: #4caf50;
  color: white;
  border: none;
  padding: 12px;
  border-radius: 4px;
  font-size: 1rem;
  font-weight: bold;
  cursor: pointer;
}
```

```
.btn-submit:hover { background-color: #43a047; }
.btn-cancel {
  background-color: #e0e0e0;
  color: #333;
  border: none;
  padding: 12px 20px;
  border-radius: 4px;
  font-size: 1rem;
  cursor: pointer;
}
.btn-cancel:hover { background-color: #d5d5d5; }

.action-buttons-gap {
  display: flex;
  justify-content: center;
  gap: 8px;
}

.btn-action-cancel {
  background-color: #ffffff;
  border: 1px solid #e53935;
  color: #e53935;
  padding: 6px 12px;
  border-radius: 4px;
  cursor: pointer;
  font-weight: bold;
  transition: all 0.2s;
}
.btn-action-cancel:hover {
  background-color: #ffebee;
}
</style>
```

適合のための改修ポイント（解説）

1. `currentYearMonth` への適合

手元のコードに合わせ、`computed` で計算を行う際に `"2026-05"` 形式の文字列を `.split('-')` して今日との差分（月数）を割り出す `getMonthOffsetFromToday` を新設しました。これにより、移動制限の判定が既存のデータモデルと完全に同期します。

2. 非活性状態でのスタイル補正

`month-selector` 内の黒～グレーのダークな色調に合わせて、非活性になった矢印ボタンが綺麗に馴染むよう `.btn-arrow:disabled` のカラーリングを調整しました。

3. `isWithinOneMonth` の日付パース

カレンダーリストに格納されている `day.recordDate` （`yyyy-MM-dd` 形式の文字列）を JavaScript の `Date` インスタンスとして安全に解釈し、時分秒を `0, 0, 0, 0` に統一することで、「ちょうど1ヶ月前の同日」から昨日までの範囲を厳密にガードします。